

HIGH AVAILABILITY WEB APPLICATION USING AWS APPLICATION LOAD BALANCER

Abstract

This project demonstrates the implementation of a High Availability Web Application architecture using Amazon Web Services (AWS). The architecture utilizes multiple Amazon EC2 instances behind an Application Load Balancer (ALB) to distribute incoming web traffic efficiently. The primary objective is to improve application availability, reliability, and fault tolerance while eliminating the single point of failure associated with traditional single-server deployments.

The solution incorporates AWS services such as Amazon EC2, Application Load Balancer, Target Groups, Security Groups, and Virtual Private Cloud (VPC). By distributing traffic among multiple backend servers and continuously monitoring their health status, the architecture ensures uninterrupted service delivery and enhanced user experience.

1. Introduction

In modern cloud computing environments, high availability is a critical requirement for web applications. Organizations must ensure that applications remain accessible even during server failures, maintenance activities, or traffic spikes.

A single-server architecture often results in a single point of failure. If that server becomes unavailable, users lose access to the application. To overcome this challenge, load balancing techniques are employed to distribute traffic across multiple servers.

Amazon Web Services provides Elastic Load Balancing services that automatically distribute incoming traffic across multiple targets, improving application performance and availability. This project demonstrates the implementation of a highly available web application using AWS Application Load Balancer and EC2 instances.

2. Project Objectives

The objectives of this project are:

- Deploy multiple Amazon EC2 instances.
- Install and configure Apache Web Server.
- Configure Security Groups for secure communication.
- Create an Internet-facing Application Load Balancer.
- Create and configure a Target Group.
- Implement Health Checks.
- Distribute incoming traffic across multiple web servers.
- Improve availability and fault tolerance.

3. AWS Services Used

3.1 Amazon EC2

Amazon EC2 provides scalable virtual machines that host the web application.

Registered EC2 Instances

- Instance ID: i-01eaf3563529a5a94
- Instance ID: i-0ccea19fa742a8802

The Apache HTTP Server was installed and configured on both instances.

3.2 Application Load Balancer (ALB)

The Application Load Balancer distributes incoming HTTP requests across multiple EC2 instances.

Load Balancer Configuration

- Name: Server-IB
- Type: Application Load Balancer
- Scheme: Internet-facing
- IP Address Type: IPv4

The load balancer acts as the entry point for all incoming web traffic.

3.3 Target Group

A Target Group was created to manage and monitor backend EC2 instances.

Target Group Details

- Name: target-IB
- Target Type: Instance
- Protocol: HTTP
- Port: 80

The target group ensures traffic is routed only to healthy instances.

3.4 Security Group

A Security Group was created to control inbound and outbound traffic.

Security Group Details

- Name: Server_IB_SG
- Description: Security Group For IB

Inbound Rule

Type	Protocol	Port	Source
------	----------	------	--------

HTTP	TCP	80	0.0.0.0/0
------	-----	----	-----------

This configuration allows users to access the application through a web browser.

3.5 Virtual Private Cloud (VPC)

All resources were deployed within a dedicated VPC.

VPC Information

- VPC ID: vpc-0e37d443172098d24

The VPC provides network isolation, routing, and security controls for AWS resources.

4. System Architecture

The architecture consists of the following components:

- Internet Users
- Application Load Balancer (Server-IB)
- Target Group (target-IB)
- Two EC2 Web Servers
- Security Group
- VPC

Architecture Flow

Users → Internet → Application Load Balancer → Target Group → EC2 Instances

When a user sends an HTTP request, the Application Load Balancer receives the request and forwards it to one of the healthy EC2 instances registered within the Target Group.

This architecture improves reliability by distributing traffic among multiple servers.

5. Health Check Configuration

Health checks continuously monitor the availability of backend servers.

Health Check Details

- Protocol: HTTP
- Health Check Path: /
- Health Check Port: traffic-port
- Interval: 30 Seconds
- Timeout: 5 Seconds
- Healthy Threshold: 5
- Unhealthy Threshold: 2
- Success Code: 200

If an instance fails a health check, the load balancer automatically removes it from traffic routing until it becomes healthy again.

6. Implementation Procedure

Step 1: Create AWS Environment

Configured AWS resources within a dedicated VPC.

Step 2: Launch EC2 Instances

Created two EC2 instances and prepared them for web hosting.

Step 3: Install Apache Web Server

Installed and configured Apache Web Server on both instances.

Commands Used:

```
sudo yum update -y
```

```
sudo yum install httpd -y
```

```
sudo systemctl start httpd
```

```
sudo systemctl enable httpd
```

Step 4: Configure Security Group

Created Security Group Server_IB_SG and allowed HTTP traffic on port 80.

Step 5: Create Target Group

Created target-IB using HTTP protocol and port 80.

Step 6: Register Targets

Registered both EC2 instances with the Target Group.

Step 7: Create Application Load Balancer

Created an Internet-facing Application Load Balancer named Server-IB.

Step 8: Configure Listener Rules

Configured HTTP listener on port 80 and associated it with target-IB.

Step 9: Verify Load Balancing

Verified successful communication between the load balancer and registered targets.

7. Results

The Application Load Balancer named Server-IB was successfully deployed and configured. The Target Group target-IB was associated with two EC2 instances and health checks were enabled.

The load balancer successfully distributed incoming traffic across registered instances while monitoring their health status.

Project Achievements

- Successfully deployed two EC2 web servers.
- Installed and configured Apache Web Server.
- Created and configured Application Load Balancer.
- Configured Security Groups.
- Created and configured Target Group.
- Implemented Health Checks.
- Distributed traffic between backend servers.
- Improved application availability and reliability.

Outcome

The project demonstrated how AWS Application Load Balancer can effectively improve web application availability, reliability, and fault tolerance through intelligent traffic distribution.

8. Advantages

- Eliminates Single Point of Failure
- Improves Application Availability
- Provides Traffic Distribution
- Enhances Reliability
- Supports Scalability
- Improves User Experience
- Enables Fault Tolerance

9. Future Scope

Future enhancements may include:

- Auto Scaling Groups
- Multi-AZ Deployment
- Route 53 DNS Integration
- HTTPS Implementation using SSL/TLS Certificates
- AWS CloudWatch Monitoring
- AWS WAF Security Protection
- CI/CD Pipeline Integration
- Containerization using Docker

10. Skills Demonstrated

- AWS EC2
- Application Load Balancer
- Target Groups
- Security Groups
- Linux Administration
- Apache Web Server
- Cloud Computing
- Networking Fundamentals
- High Availability Architecture

11. Conclusion

This project successfully implemented a High Availability Web Application using AWS Application Load Balancer and multiple Amazon EC2 instances. The architecture distributed incoming traffic across multiple servers while continuously monitoring instance health. By eliminating the single point of failure and introducing load balancing, the solution improved availability, reliability, and fault tolerance.

The project provided valuable hands-on experience with core AWS services and demonstrated real-world cloud architecture principles commonly used in enterprise environments.